

Program 1: One Process (Before fork())

```
#include <stdio.h>
#include <unistd.h>
int main()
{
    printf("PID = %d\n", getpid());
    return 0;
}
```

Only one process exists.
Introduce getpid().
|

```
prajwal@PRAJWAL:~/os/fork$ gcc program1.c -o program1
prajwal@PRAJWAL:~/os/fork$ ./program1
PID = 793
```

Program 2: fork() Creates a Child

```
#include <stdio.h>
#include <unistd.h>
int main()
{
    fork();
    printf("Hello\n");
    return 0;
}
```

```
prajwal@PRAJWAL:~/os/fork$ nano program2.c
prajwal@PRAJWAL:~/os/fork$ gcc program2.c -o program2
prajwal@PRAJWAL:~/os/fork$ ./program2
Hello
Hello
```

Program 3: Parent vs Child

```
#include <stdio.h>
#include <unistd.h>
int main()
{
    pid_t pid;
    pid = fork();
    if(pid == 0)
        printf("I am Child\n");
    else
        printf("I am Parent\n");
    return 0;
}
```

```
prajwal@PRAJWAL:~/os/fork$ gcc program3.c -o program3
prajwal@PRAJWAL:~/os/fork$ ./program3
I am Parent
I am Child
prajwal@PRAJWAL:~/os/fork$ |
```

Program 4: Display Both PIDs

```
#include <stdio.h>
#include <unistd.h>
int main()
{
    pid_t pid;
    pid = fork();
    if(pid == 0)
    {
        printf("Child PID = %d\n", getpid());
        printf("Parent PID = %d\n", getppid());
    }
    else
    {
        printf("Parent PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
    }
    return 0;
}
```

```

prajwal@PRAJWAL:~/os/fork$ gcc program4.c -o program4
prajwal@PRAJWAL:~/os/fork$ ./program4
Parent PID = 976
Child PID = 977
Child PID = 977
Parent PID = 976
prajwal@PRAJWAL:~/os/fork$ |

```

Program 5: Parent Waits for Child

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main()
{
    pid_t pid = fork();
    if(pid == 0)
    {
        printf("Child Executing\n");
    }
    else
    {
        wait(NULL);
        printf("Parent Executing\n");
    }
    return 0;
}

```

```

prajwal@PRAJWAL:~/os/fork$ gcc program5.c -o program5
prajwal@PRAJWAL:~/os/fork$ ./program5
Child Executing
Parent Executing
prajwal@PRAJWAL:~/os/fork$ |

```

Program 6: Multiple fork() Calls

```

#include <stdio.h>
#include <unistd.h>
int main()
{
    fork();
    fork();
    printf("PID = %d\n", getpid());
    return 0;
}

```

```
prajwal@PRAJWAL:~/os/fork$ gcc program6.c -o program6
prajwal@PRAJWAL:~/os/fork$ ./program6
PID = 1061
PID = 1062
PID = 1064
PID = 1063
prajwal@PRAJWAL:~/os/fork$ |
```

Program 7: Child Executes Another Program (exec())

```
#include <stdio.h>
#include <unistd.h>
int main()
{
    pid_t pid = fork();
    if(pid == 0)
    {
        printf("Child executing ls...\n");
        execl("/bin/ls", "ls", "-l", NULL);
    }
    else
    {
        printf("Parent Waiting...\n");
    }
    return 0;
}
```

```

prajwal@PRAJWAL:~/os/fork$ gcc program7.c -o program7
prajwal@PRAJWAL:~/os/fork$ ./program7
Parent Waiting...
Child executing ls...
prajwal@PRAJWAL:~/os/fork$ total 156
-rwxr-xr-x 1 prajwal prajwal 16000 Aug  3 04:19 a.out
-rwxr-xr-x 1 prajwal prajwal 16000 Aug  3 04:16 program1
-rw-r--r-- 1 prajwal prajwal   99 Aug  3 04:15 program1.c
-rwxr-xr-x 1 prajwal prajwal 16000 Aug  3 04:17 program2
-rw-r--r-- 1 prajwal prajwal   94 Aug  3 04:18 program2.c
-rwxr-xr-x 1 prajwal prajwal 16000 Aug  3 04:19 program3
-rw-r--r-- 1 prajwal prajwal  163 Aug  3 04:19 program3.c
-rwxr-xr-x 1 prajwal prajwal 16088 Aug  3 04:20 program4
-rw-r--r-- 1 prajwal prajwal  278 Aug  3 04:20 program4.c
-rwxr-xr-x 1 prajwal prajwal 16040 Aug  3 04:21 program5
-rw-r--r-- 1 prajwal prajwal  214 Aug  3 04:21 program5.c
-rwxr-xr-x 1 prajwal prajwal 16040 Aug  3 04:24 program6
-rw-r--r-- 1 prajwal prajwal  116 Aug  3 04:23 program6.c
-rwxr-xr-x 1 prajwal prajwal 16040 Aug  3 04:26 program7
-rw-r--r-- 1 prajwal prajwal  220 Aug  3 04:25 program7.c

```

Program 10: fork() Inside a Loop (Fun)

```

#include <stdio.h>
#include <unistd.h>
int main()
{
    int i;
    for(i=0;i<3;i++)
        fork();
    printf("PID = %d\n",getpid());
    return 0;
}

```

```

prajwal@PRAJWAL:~/os/fork$ gcc program10.c -o program10
prajwal@PRAJWAL:~/os/fork$ ./program10
PID = 1174
PID = 1175
PID = 1177
PID = 1176
PID = 1178
PID = 1179
PID = 1180
PID = 1181
prajwal@PRAJWAL:~/os/fork$ |

```